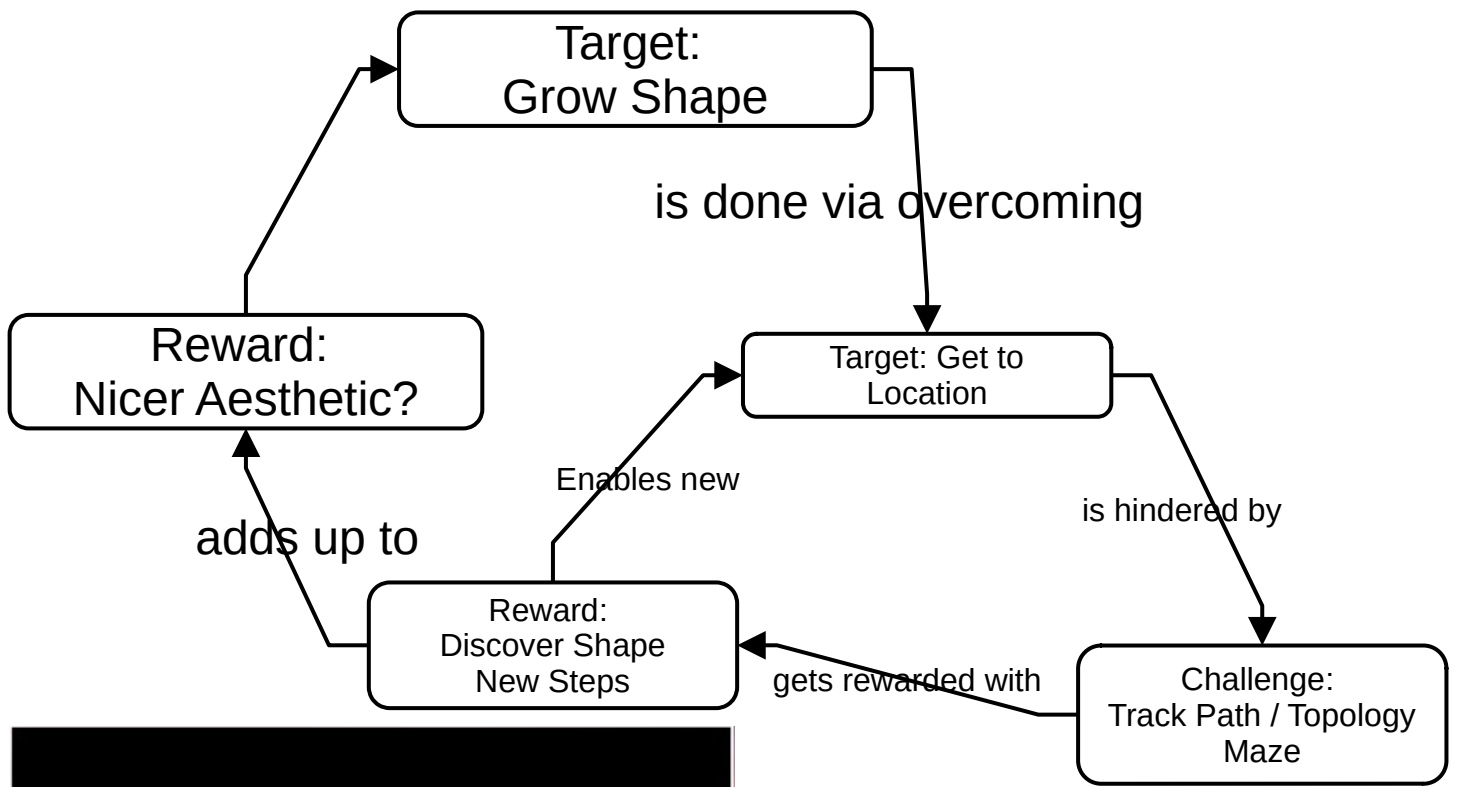
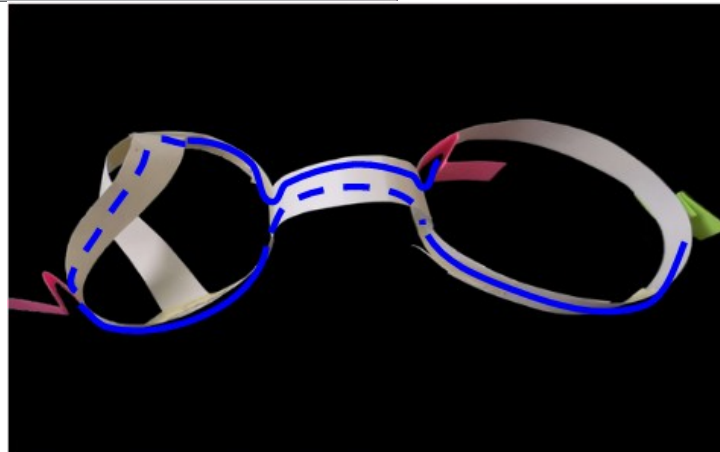




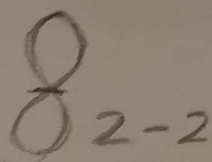
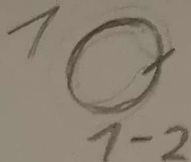
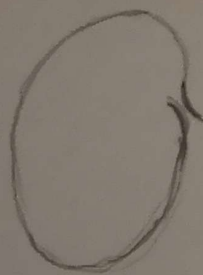
Start at Green,  
get to pink



Find an actual  
Path

Get rewarded with new  
shape & new targets to  
go to





2-2

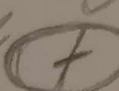
3

Through  
The loops



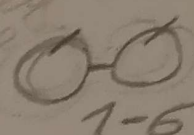
2-2

Merged  
Möbi

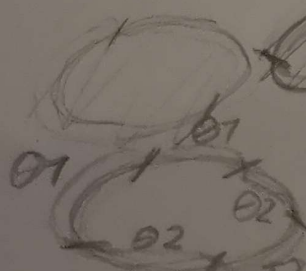


2-2

Basically  
the same

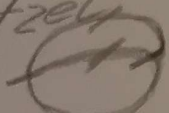


1-6

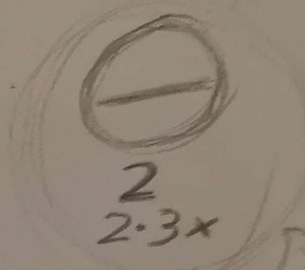


not Equal  
Butterfly

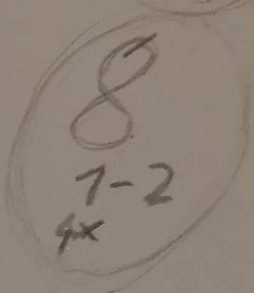
Pretzel



1  
2x3x

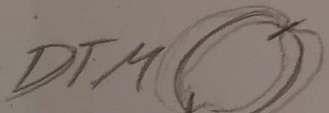


2  
2x3x

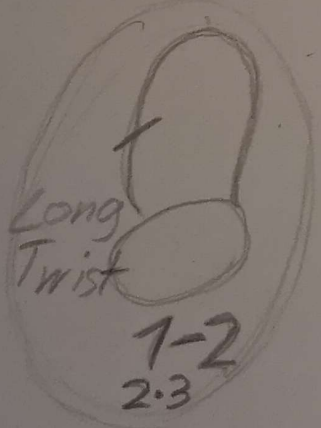


1-2  
4x

one  
of  
them

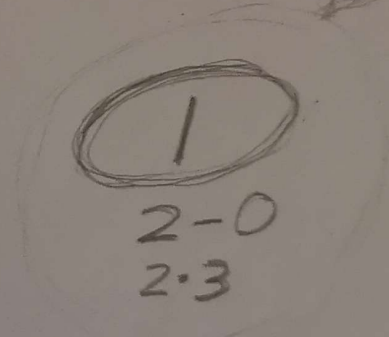


2-2



Long  
Twist

1-2  
2x3



1  
2-0  
2x3

- 1 Möbius
- 2 Figure 8
- 3 Merged Möbius
- 4 Butterfly
- 5 Circular
- 6 Pretzel
- 7 Long Twist
- 8 Twisted Heart

Switched to  
counting twists & crossings

- 1 1-1  $\rightarrow 0$
- 2 2-0  $\rightarrow 1 \times 4+$
- 3 2-1  $\rightarrow 0$
- 4 1-2/3  $\rightarrow 0$
- 5 2-0  $\rightarrow 2 \times 3+$
- 6 1-2  $\rightarrow 2 \times 3+$
- 7 1-1  $\rightarrow 2 \times 3+$
- 8 4-0  
3-1

## General Note

I have named the shapes based on the first things that came to my mind upon looking at them; They don't mean anything in particular & have nothing to do with their designed gameplay purpose or what they should look like;

So if you want to take inspiration from the names, then do it, but also feel free to ignore the names;

Möbius





Figure-8

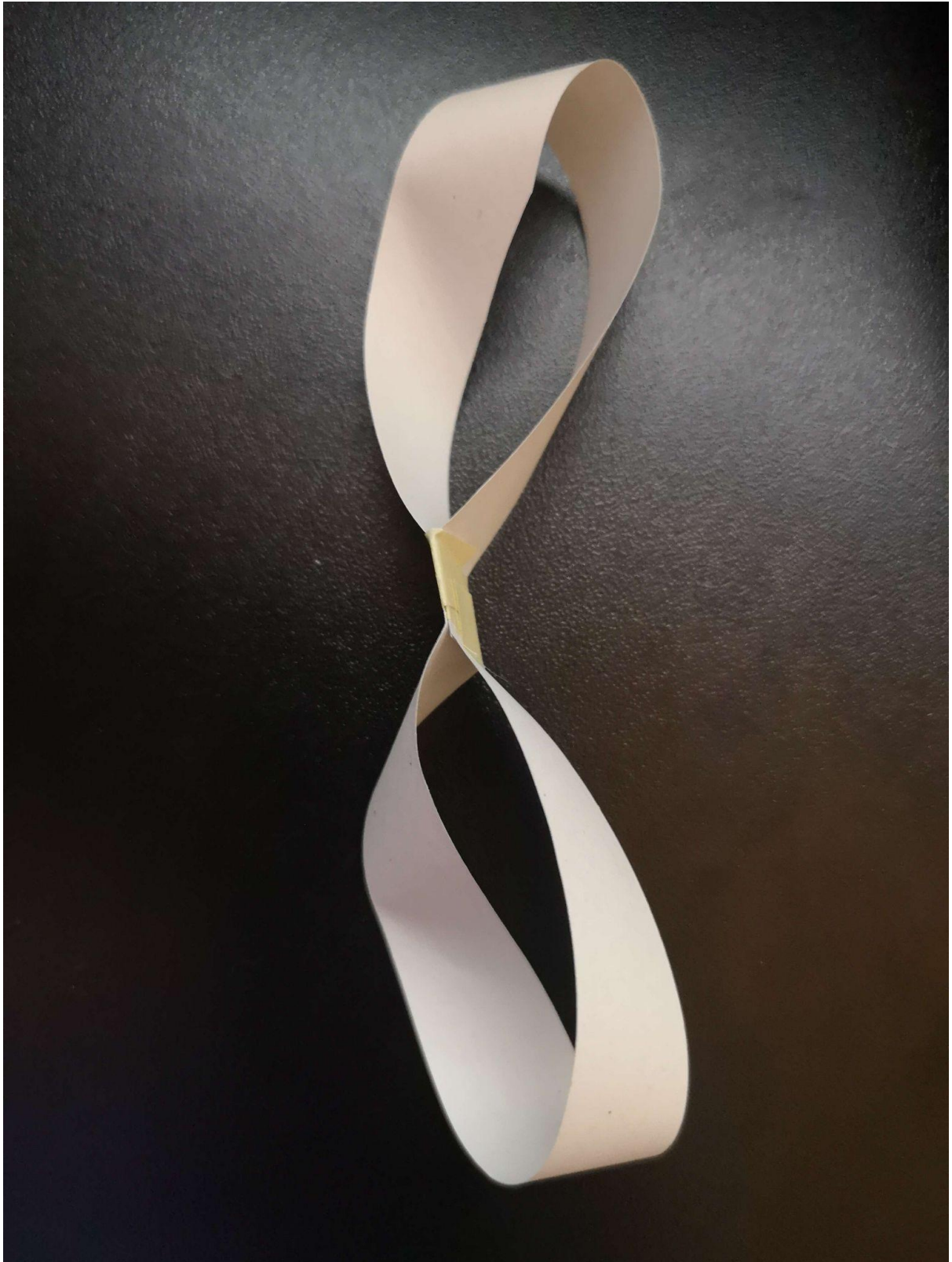


## Through the Loop





Butterfly

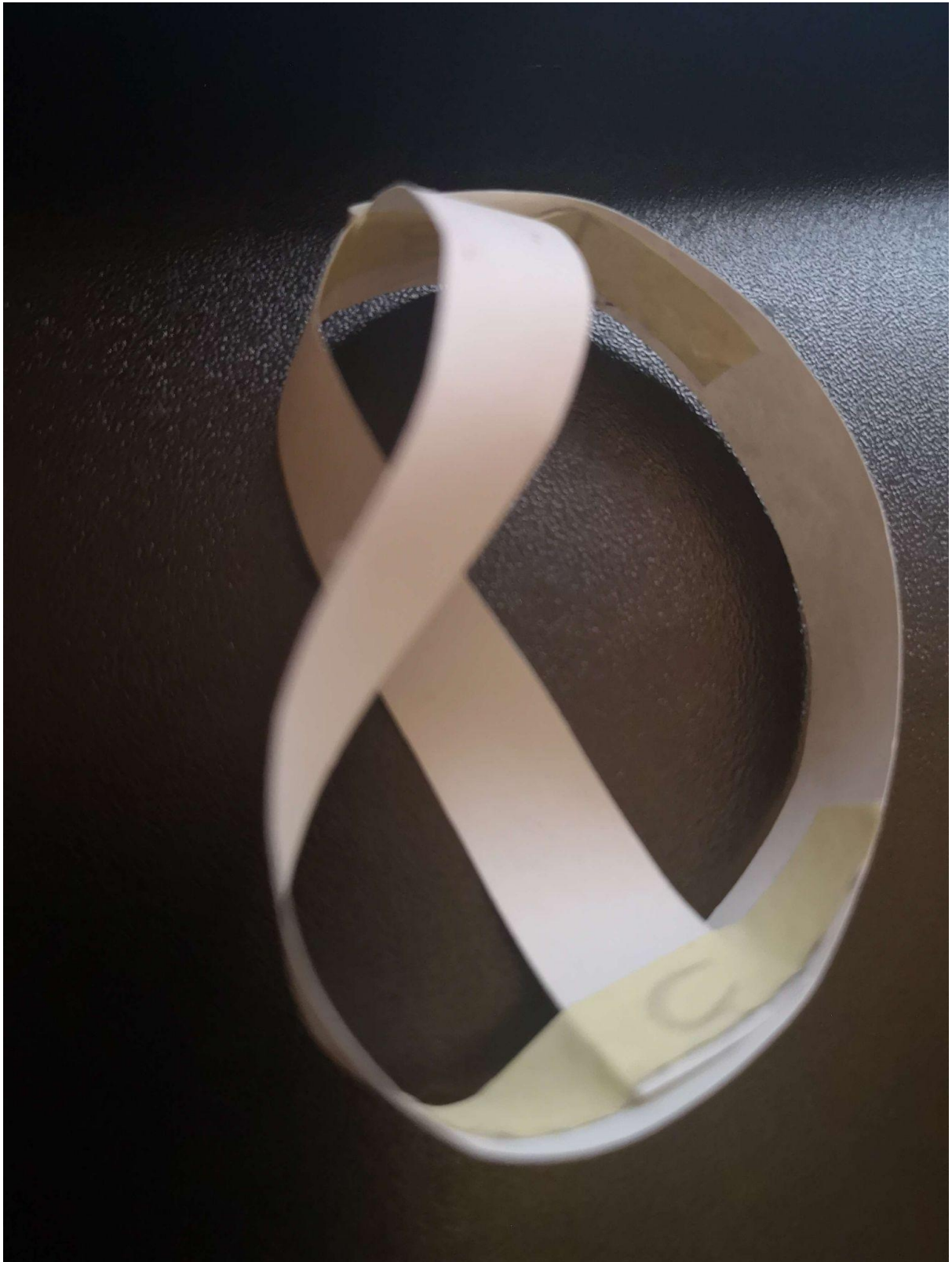


## Roundabout

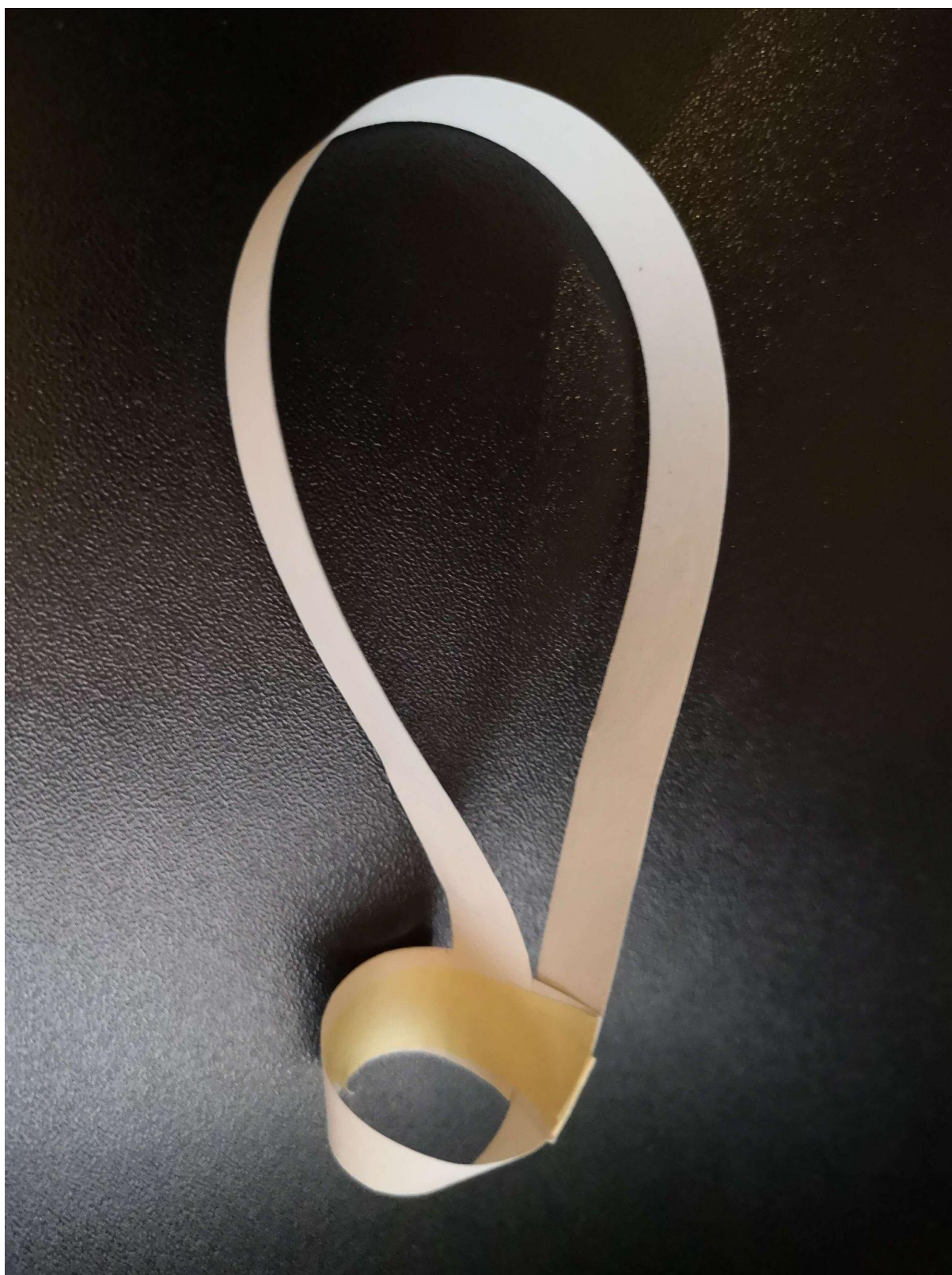




Pretzel



Skyscraper





## Twisted Heart

I had some issues making this out of paper, therefore I have two images for you, that are partially correct



This one is the topologically correct ones; It's kind of like 3 circles which overlapping parts & a twist in the connected part of the top 2



This is how it should look like in terms of how curvy it should be; however this lacks the twist on the top connection; (& also, the edge on the bottom should be a bit of a rounded edge)



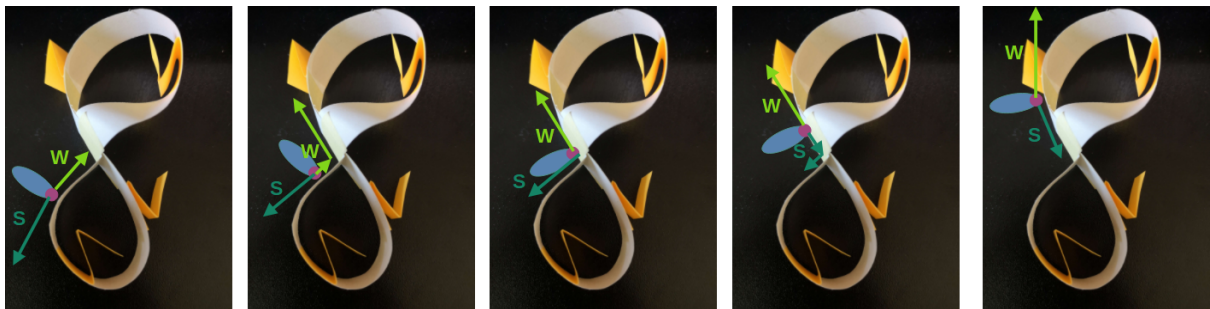
## Movement Basics - Explanation:

The player (blue) should be able to walk along the bend surface underneath them. The direction they are facing is controlled via the mouse, their movement via WASD-keys. The player can not walk over edges, so if they are at an edge & try to walk further in that direction, nothing happens.

Also, use whatever collider-shape makes it easiest to move the character around in this way, because there won't be a moment in the game, where the precise collision will be important.

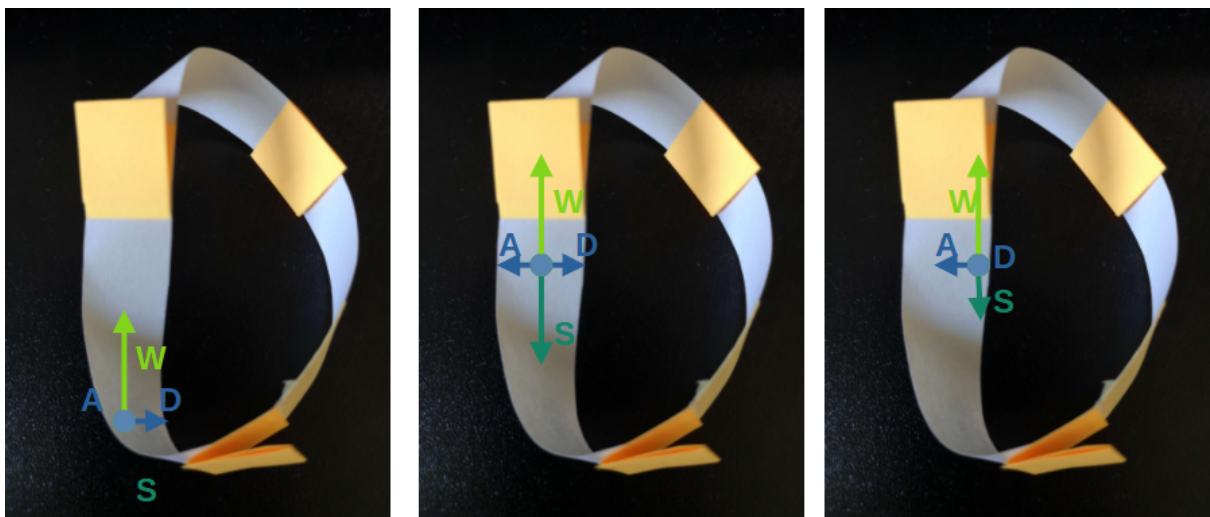
## Example:

### Sideways Perspective:



Between the second and third image the player walks around a rather sharp corner (I don't expect any of these super sharp corner to be in the game). So if you find a good solution for these  $>45^\circ$  rotations, than go for it, otherwise it's not important if it bugs with super sharp corners; just make sure that the movement on convex as well as concave surfaces works.

### Topdown Perspective:



in the first image pressing 'A' or 'S' does nothing as the player faces on their back-left side.  
in the third image the same goes for pressing 'D'.

### Variables:

- movespeed (5f)
- 

### Implementation Suggestions / pseudo-code:

```
if (movement.Button)
{
    push player towards direction via physics()
    Collision subPlayerCol = check what is under the player from playerchar perspective
    push player to subPlayerCol
    player rotation = subPlayerCol.normal
}
```

### Reference:

<https://www.youtube.com/watch?v=OcvD8BXars8>  
<https://www.youtube.com/watch?v=QSDUA9YpVwQ>

## Movement Ideas:

WASD - Direct Controls

WASD - Momentum - Spinning

Dashing / Sprinting towards the next way-crossing

Spinning the whole environment

Paths that characters walk along

- > drawing full Paths

- > setting Points that characters should walk towards

- > points on the map the player can only click on these to make a character move towards them

Ability-based -> players have to collect different abilities throughout the map, they can only move by using these abilities

Sign-based -> The player sets up sign at the crossings, the characters look at these and walk according to them

Warp-Diving = The MC forms itself into a floating ball (with a salto or something like that) & can then go highspeed / slow down (potentially in combination with an energy level for Warp-Dives, which can be replenished with the rewards)

Swim-Through the Buildings = The Character jumps into the ground & merges with it (becomes a Sprite) the following dash-swim enables it to quickly travel from one place to another, until the character smashes into a stopping point

Examples / Tutorials:

<https://www.youtube.com/watch?v=OcvD8BXars8>

<https://www.youtube.com/watch?v=QSDUA9YpVwQ>

<https://youtu.be/JvbsktWVHU8?t=17> (zelda / tutorial for merging character into a Wall

<https://youtu.be/prChg79-fo?t=233> (Samus Ball-Morph & swims in lava)



## Potentially needed Features:

- Switching Camera from first to third-person camera
- Move with the character
- rotate the camera

## Open Questions / possible Solutions:

### When to switch camera

- Anytime via Scrollwheel
- Anytime via Button & Scrollwheel -> how to move camera back to player perspective from godmode cam & where does godmode cam get placed upon switching to it
  - Button that Lerps between camera positions
  - Timer in the background, that while in player perspective, lerps the godmode cam back to be behind player
  - Timer, that after a certain time (in player perspective) resets the godmode cam
  - map positions have a declared starting godmode cam position
- Map-Locations
  - how to split them
  - map positions have a declared starting godmode cam position
- via Item
  - where to collect

### How to Rotate the camera

- Fixed behind the character
- Added up & down rotation -> how far up & down?
  - limit by  $0^\circ$  (=ground) and  $90^\circ$  (directly from the top) (or some other limitation)
  - completely around
- Left-Right rotation independent of character -> How to move character
  - W/S to move along surface (in the direction the player is facing) & additional keys for rotating the character around
  - Added keys for rotating the character to different directions
  - Character rotates head towards the direction the camera is facing, but not their body
- 

## Order of trying solutions:

### First solution

- Scrollwheel
- reset to first-person button
- resimplify movement (= character can only be rotated along the strips; at crossing the character there is a rotate-around button)

### Improvements that most likely help the game

- keeping two switchable cams
- Map-Points

## Explanation:

Shapes have spawning spots placed on them.

Whenever one of their triggers (e.g. 'Start of Game', 'Player collected Item', 'Player took X Step' ...) gets activated, they call their checks (one check per item type) whether they should spawn that specific item.

If the player collides with an item, they collect it (=their counter for the item increases by one).

Description of items soon in other documents.



### Example:

The player (blue oval) just collected an item at the spawning spot on the top-left. This triggers the spawning check of all spawning spots on the map. The top-right spawning spot calls its check function whether it should spawn a new ItemType-A; For now the check is a simple random chance of  $[\frac{1}{4}]$ ; It returns false; It continues with ItemType-B (chance  $[\frac{1}{3}]$ ) and also returns false. There are no more checks left. The next spawning spot does its checks & so they continue one after another. The bottom right spawning spot spawns an ItemType-A.

(All checks are performed within the same frame of the trigger.)



The Player starts walking towards the newly spawned item. As they touch it, the item vanishes, their counter for ItemType-A increases by 1 and a new item-spawn-check for each spawning spot is triggered.

## Variables:

- Position of the spawning spots
  - triggers for the spawning check (will be added later.)
  - probability of spawning (/algorithms for calculating will be added later. )
  - time between checks of spawning new reward

## Implementation suggestion / Pseudocode:

REWARD-SPAWNER

```
{{  
spawnTimer = 3 sec
```

Start

```
{  
    Invoke & Repeat every spawnTimer seconds(CheckSpawn)  
}
```

CheckSpawn

```
{  
    if (checkspawnAlgorithm == true)  
    {  
        SpawnItem()  
    }  
}  
}}
```

REWARD

```
{{  
if (touchesPlayer)  
{  
    player.plusReward() //(should be as a function because we will add more to the  
moment once the player collects a reward later)  
    Destroy(this.gameObject)  
}  
}}
```

PLAYER

```
{{  
  
variable rewardAmount = 0;  
  
plusReward  
{  
    rewardAmount += 1;  
}  
}}
```



## Ideas

Spawning Locations of Rewards -> Graphical Idea?

Reward Types:

- new Shape
- (new Shape-Part)
- Shape-Repair (if broken shape system)
- Map-Look
- Orientation Point
- One-Time-Borrow / One-Time Edge-Walk
- Reward-Dispenser
- Extra Steps / Dives
- Extra Energy
- Extra Bridges

Spaces where the player can look at the map

Orientation Points / Ornament

Fast-Travel Track

Potentially needed Features:

New Shape = Reward-Dispenser

Map-Points = Orientation-Points

Steps

Bridges

Open Questions / possible Solutions:

**How to distribute items over shapes (presets, actual ones, placed ones) & balance?**

- via algorithm for each item spawning chance
- via algorithm called open collecting a reward-dispenser
- add timers on shapes
  - include something timer-wise in regards to the shape-form

**How to avoid the game becoming kind of impossible as overall shape size increases?**

- ???

**Which step limiter?**

- energy
  - use X for starting / stopping dive
    - regain energy for well-timed stopped warpdive -> how to integrate this with the actual experience of the shape?
  - use Y/time of dive

## Order of trying solutions:

### First Solution:

- Adding new shapes dispenses new rewards over the map
  - via each item-spawner checking it's chances
- Automatically new energy & bridge-items on the new shape

### Improvements that should be tried:

-